# LaTeX verbatim 与 verbatim* 环境的新实现

Rainer Schöpf　　Bernd Raichle　　Chris Rowley

2023/11/06

This file is maintained by the LaTeX Project team.
Bug reports can be opened (category tools) at
https://latex-project.org/bugs.html.

**本宏包出于稳定性考虑仍保留在 LaTeX tools 宏包集合中。虽然会继续修复 verbatim 的 bug，但不再考虑添加新功能。对于新的需求，LaTeX 团队建议使用 fancyvrb 代替 verbatim。**

**摘要**

本宏包重新实现了 LaTeX 的 verbatim 和 verbatim* 环境。此外它还提供了一个 comment 环境，用于跳过\begin{comment} 与下一个\end{comment} 之间的所有命令或文本。它还定义了\verbatiminput 命令，用于以 verbatim 方式包含整个文件。

## 1 使用说明

LaTeX 的 verbatim 和 verbatim* 环境有几个可能引发问题的特性：

- 由于检测\end{verbatim} 的方式（即宏参数定界），你不能在\end 记号与 {verbatim} 之间留空格。
- 由于 TeX 必须先读取\begin{verbatim} 与\end{verbatim} 之间的全部文本才能输出任何内容，过长的 verbatim 列表可能会耗尽 TeX 的内存。

这两点中，前者只能算小麻烦，后者却是真正的限制。

本宏包文件包含了对 verbatim 和 verbatim* 环境的重新实现，克服了上述限制。不过新旧实现之间有一处不兼容：旧版本会把\end{verbatim} 命令同一行上的文本当作单独一行处理。

**新版本则会直接忽略这些文本。**

（这是消除旧 verbatim 环境大小限制所必须付出的代价。）不过它会发出如下形式的警告信息：

```
LaTeX warning: Characters dropped after \end{verbatim*}!
```

这并不是真正的问题，因为这些文本很容易移到下一行而不影响输出。

新实现还解决了上面提到的第二个问题：现在可以在\end 与 {verbatim} 或 {verbatim*} 之间留空格（但不能开始新行）：

```
\begin {verbatim*}
   test
   test
\end {verbatim*}
```

此外我们引入了一个 comment 环境，其效果是：\begin{comment} 与\end{comment} 之间的文本无论长什么样都被直接忽略。乍看之下这与 verbatim 列表的用途大不相同，但实际上这两个概念的实现非常相似。二者都依赖于这样一个事实：\begin{...} 与\end{...} 之间的文本由 TEX 读取，不解释任何命令或特殊字符。verbatim 与 comment 之间剩下的唯一区别是：前者的文本要排版出来，后者的文本则被丢弃。注意这些环境不能嵌套。

\verbatiminput 是一个带一个参数的命令，用于以 verbatim 方式包含一个文件；也就是说，命令\verbatiminput{xx.yy} 的效果等价于

```
\begin{verbatim}
⟨文件 xx.yy 的内容⟩
\end{verbatim}
```

该命令还有一个 *-变体，会把空格打印为 ␣。

## 2 面向宏包作者的接口

LATEX 2ε 的 verbatim 环境没有为程序员提供良好的接口。与之不同，本宏包提供了一个简单的机制来实现类似功能，这里实现的 comment 环境就是一个示例，展示了能做什么以及怎么做。

## 2.1 简单示例

现在可以使用 verbatim 环境来定义你自己的环境了。例如，

```
\newenvironment{myverbatim}%
  {\endgraf\noindent MYVERBATIM:%
   \endgraf\verbatim}%
  {\endverbatim}
```

之后就可以像 verbatim 环境一样使用它，即

```
\begin {myverbatim}
  test
  test
\end {myverbatim}
```

另一种用法是写

```
\let\foo=\comment
\let\endfoo=\endcomment
```

此后环境 foo 就与 comment 环境相同，即其内部的所有内容都被忽略。

你还可以在\verbatim 宏被调用之后添加特殊命令，例如

```
\newenvironment{myverbatim}%
  {\verbatim\myspecialverbatimsetup}%
  {\endverbatim}
```

不过此时你可能想了解下面要讲的\every@verbatim 钩子。不过仍有若干限制：

1. 你不能在定义内部使用\begin 或\end 命令，例如下面两个示例不能工作：

   ```
   \newenvironment{myverbatim}%
   {\endgraf\noindent␣MYVERBATIM:%
   ␣\endgraf\begin{verbatim}}%
   {\end{verbatim}}
   \newenvironment{fred}
   {\begin{minipage}{30mm}\verbatim}
   {\endverbatim\end{minipage}}
   ```

   如果你尝试这些示例，TEX 会报 “runaway argument” 错误。更一般地说，在新环境的定义中不能使用\begin …\end 或相关环境。正确的定义方式应该是

   ```
   \newenvironment{fred}
   ```

```
{\minipage{30mm}\verbatim}
{\endverbatim\endminipage}
```

2. 你不能在用户自定义的命令内部使用 verbatim 环境；例如

```
\newcommand{\verbatimfile}[1]%
␣␣␣␣␣␣␣␣␣␣␣{\begin{verbatim}\input{#1}\end{verbatim}}
```

不能工作；下面这个也不行：

```
\newcommand{\verbatimfile}[1]{\verbatim\input{#1}\endverbatim}
```

3. 新定义环境的名字不能包含字符类别不是 11（字母）或 12（其他）的字符，否则将无法工作。

## 2.2 接口

从简单的开始讲起。有时你可能需要为 verbatim 文本使用特殊字体，或者使用较小字号的计算机现代打字机字体。

你可以通过重定义宏\verbatim@font 来选择字体。该宏在每段 verbatim 文本开始时被调用，用于选择字体形状。不要把它用于其他用途；如果你发现自己在滥用它，可以去看看下面的\every@verbatim 钩子。

默认情况下，\verbatim@font 切换到打字机字体并禁用其中的连字。

有一个钩子（即记号寄存器）叫做\every@verbatim ，其内容会在每段 verbatim 文本之前被插入到 TEX 的” 嘴” 中。请使用\addto@hook 宏向这个钩子添加内容。用法如下：

\addto@hook⟨钩子名⟩{⟨要添加的命令⟩}

在完成所有特定设置（如切换字符类别）之后，会调用\verbatim@start 宏。它启动本宏包实现的扫描机制主循环。任何想利用此功能的其他环境都应把该宏作为最后一个动作来执行。

这些是与 verbatim 环境启动相关的宏。完成这些（及其他）设置后，本宏包中的代码按以下方式从输入流中读取并处理字符：

1. 在读取一行输入的第一个字符之前，执行宏\verbatim@startline 。
2. 读取若干字符后，以这些字符为唯一参数调用宏\verbatim@addtoline 。每行可能发生多次（当该行含有\end 命令时）。
3. 到达行尾时,调用宏\verbatim@processline 来处理\verbatim@addtoline 累积的字符。

4. 最后，在环境被\end 宏调用结束之前，会调用宏\verbatim@finish 。

为说明这一点，我们看一下标准 verbatim 环境。在这种情况下，上述三个宏定义如下：

1. \verbatim@startline 清空字符缓冲区（一个记号寄存器）。
2. \verbatim@addtoline 把其参数中的记号添加到字符缓冲区\verbatim@line ，不展开它们。
3. \verbatim@processline 排版缓冲区中累积的字符。

有了这些,实现 comment 环境就非常简单了:在这种情况下,\verbatim@startline 和\verbatim@processline 被定义为空操作，而\verbatim@addtoline 丢弃其参数。

让我们用这个机制来定义一个 verbatim 环境的变体，在左边距打印行号。假设用一个叫 VerbatimLineNo 的计数器来实现。假设该计数器已由环境正确初始化，那么在这种情况下\verbatim@processline 定义为:

```
\def\verbatim@processline{%
  \addtocounter{VerbatimLineNo}{1}%
  \leavevmode
  \llap{\theVerbatimLineNo\ \hskip\@totalleftmargin}%
  \the\verbatim@line\par}
```

另一种可能是定义一个 verbatim 环境的变体，把整段 verbatim 文本放入盒子并居中。注意盒子中的文本应少于一页，否则你需要修改这个示例。

```
\def\verbatimboxed#1{\begingroup
  \def\verbatim@processline{%
    {\setbox0=\hbox{\the\verbatim@line}%
     \hsize=\wd0
     \the\verbatim@line\par}}%
  \setbox0=\vbox{\parskip=0pt\topsep=0pt\partopsep=0pt
                 \verbatiminput{#1}}%
  \begin{center}\fbox{\box0}\end{center}%
 \endgroup}
```

最后一个稍有分量的示例是描述一个叫 verbatimwrite 的环境的定义。它把其内部所有文本写入一个文件，文件名作为参数给出。我们假设有一个叫\verbatim@out 的流号已经通过\newwrite 宏预留。

从宏\verbatimwrite 的定义开始。

```
\def\verbatimwrite#1{%
```

首先调用\@bsphack，使该环境不影响间距。然后打开文件并设置所有特殊字符的类别码：

```
\@bsphack
\immediate\openout \verbatim@out "#1" %
\let\do\@makeother\dospecials
\catcode`\^^M\active
```

下面几个宏的默认定义

```
\verbatim@startline
\verbatim@addtoline
\verbatim@finish
```

也在这个环境中使用。只需要在调用\verbatim@start 之前修改宏\verbatim@processline：

```
\def\verbatim@processline{%
  \immediate\write\verbatim@out{\the\verbatim@line}}%
\verbatim@start}
```

\endverbatimwrite 的定义非常简单：关闭流并调用\@esphack 来恢复正确间距。

```
\def\endverbatimwrite{\immediate\closeout\verbatim@out\@esphack}
```

# 3 实现

我们做的第一件事是确保本文件不会被读取两次。为此我们检查宏\verbatim@@@是否已定义。如果已定义，就停止读取本文件。这里的" 宏包" 守卫使得在抽取用于测试本宏包的 driver 文件时，大部分代码可以被排除。

```
1 ⟨*package⟩
2 \NeedsTeXFormat{LaTeX2e}
3 \ProvidesPackage{verbatim}
4      [2026-08-17 v1.5z LaTeX2e package for verbatim enhancements]
5 \@ifundefined{verbatim@@@}{}{\endinput}
```

我们使用与$\mathcal{AMS}$-TEX中\comment …\endcomment 宏类似的机制：一次读取一行，检查它是否包含\end{...} 记号。然后就可以判断是否到达了 verbatim 文本的末尾，还是需要继续。

### 3.1 预备工作

\every@verbatim 钩子（即记号寄存器）\every@verbatim 初始化为 ⟨空⟩。

```
\newtoks\every@verbatim
\every@verbatim={}
```

\@makeother \@makeother 以一个字符为参数，把它的类别码改为 12（其他）。

```
\def\@makeother#1{\catcode`#112\relax}
```

\@vobeyspaces 宏\@vobeyspaces 使输入中的空格在输出中打印为空格。

```
\begingroup
\catcode`\ =\active%
```

因为空格是活动字符，我们无法把下面的代码整齐缩进——否则行首的空格会被当作 \@ifl@t@r 的第三个和第四个参数。

```
\@ifl@t@r\fmtversion{2023-11-01}%
{\def\x{\def\@vobeyspaces{\catcode`\ \active\let \@xobeysp\@vobeytabs}}}%
{\def\x{\def\@vobeyspaces{\catcode`\ \active\let \@xobeysp}}}%
\expandafter\endgroup\x
```

\@xobeysp 宏\@xobeysp 在输出中产生恰好一个空格，并防止在它之前断行。（\@M 是数字 10000 的缩写。）

```
\def\@xobeysp{\leavevmode\penalty\@M\ }
```

\verbatim@line 我们使用一个新定义的记号寄存器\verbatim@line 作为字符缓冲区。

```
\newtoks\verbatim@line
```

下面四个宏以适合 verbatim 和 verbatim* 环境的方式全局定义。

\verbatim@startline \verbatim@startline 通过清空字符缓冲区（\verbatim@line）来开始处理
\verbatim@addtoline 一行。
\verbatim@processline

```
\def\verbatim@startline{\verbatim@line{}}
```

\verbatim@addtoline 把其参数中的记号添加到缓冲区寄存器\verbatim@line，不展开它们。

```
\def\verbatim@addtoline#1{%
  \verbatim@line\expandafter{\the\verbatim@line#1}}
```

在 verbatim 或 verbatim* 环境中处理一行意味着打印它。结束一行意味着我们必须开始新段落。我们用\par 来做这件事。注意\par 在\@verbatim 中被重定义，以迫使 TEX 进入水平模式并插入一个空盒子，这样输入中的空行才会在输出中出现。

```
\def\verbatim@processline{\the\verbatim@line\par}
```

\verbatim@finish 默认情况下，\verbatim@finish 处理剩余字符。调用这个宏时我们面临如下问题：当遇到\end{verbatim} 命令时，\verbatim@processline 被调用来处理该命令同一行上前面的字符。如果没有字符，而我们不检查这种情况，就会输出一个空行。

如果行为空，\the\verbatim@line 展开为空。为检测这一点，我们使用一个与 TEXbook 第 376 页类似的技巧，但用 $...$ 代替那里的! 记号。这些 $ 记号的类别码永远不可能与记号寄存器\verbatim@line 中可能出现的 $ 记号相同，因为后者总是以类别码 12（其他）读取的。注意\ifcat 会展开后面的记号，使\the\verbatim@line 被累积的字符替换。

```
\def\verbatim@finish{\ifcat$\the\verbatim@line$\else
  \verbatim@processline\fi}
```

## 3.2 verbatim 与 verbatim* 环境

\verbatim@font 我们首先定义宏\verbatim@font ，用于选择字体并设置与字体相关的参数。然后展开\@noligs （定义在 LaTeX $2_\varepsilon$ 内核中）。它除了做其他事之外，会遍历\verbatim@nolig@list 以避免某些连字。\verbatim@nolig@list 是 LaTeX $2_\varepsilon$ 内核中定义的宏，展开为

```
\do\`\do\<\do\>\do\,\do\'\do\-
```

这个列表中的所有字符在某种字体中都可能成为连字的一部分。

```
\def\verbatim@font{\normalfont\ttfamily
                   \hyphenchar\font\m@ne
                   \@noligs}
```

\@verbatim 宏\@verbatim 负责做好各项设置。首先，插入\every@verbatim 钩子的记号。然后启动一个 trivlist 环境并插入其第一个\item 命令。verbatim 或 verbatim* 环境的每一行都将被当作独立段落处理。

```
\def\@verbatim{\the\every@verbatim
  \trivlist \item \relax
```

下面这段额外的垂直间距是为了与 LaTeX 内核兼容：否则，使用本 verbatim 宏包会改变嵌套在 quote 环境中的 verbatim 环境的垂直间距。

```
  \if@minipage\else\vskip\parskip\fi
```

段落参数被适当设置：环境开头的惩罚值、左右边距、段落缩进、填充最后一行的胶值，以及段落之间的垂直间距。后者必须为零，因为我们不希望在行与行之间添加额外间距。

```
  \@beginparpenalty \predisplaypenalty
  \leftskip\@totalleftmargin\rightskip\z@
```

```
\parindent\z@\parfillskip\@flushglue\parskip\z@
```

这里有一点要说明：list 环境使用 TEX 的\parshape 原语来为列表第一行设置特殊缩进。如果列表以 verbatim 环境开头，这个\parshape 仍然生效。因此我们必须显式重置这个内部参数。我们可以通过把 0 赋给\parshape 来做到。不过有一个更简单的办法：直接告诉 TEX 开始新段落。正如 TEXbook 第 103 页所述，这会把\parshape 重置为零。

```
\@@par
```

现在我们确保\par 有正确的定义，即迫使 TEX 进入水平模式并包含一个空盒子。这是为了确保空行确实在输出中出现。之后我们插入\interlinepenalty，因为 TEX 不会主动在段落（这里是行）之间添加惩罚值。否则即使存在\samepage 声明，verbatim 环境也可能被跨页拆开。

不过，在顶对齐的 minipage 中，这会导致顶部多出一个空行。因此需要一个稍复杂的结构。这里重要的一点是把\leavevmode 作为第一行的第一个宏，例如当列表项的第一个内容就是一个空 verbatim 行时。

```
\def\par{%
  \if@tempswa
    \leavevmode\null\@@par\penalty\interlinepenalty
  \else
    \@tempswatrue
    \ifhmode\@@par\penalty\interlinepenalty\fi
  \fi}%
```

为了在环境不包含任何文本时避免错误信息，我们重定义\@noitemerr，它在这种情况下会被\endtrivlist 调用。

```
\def\@noitemerr{\@warning{No verbatim text}}%
```

现在调用\obeylines 使行尾字符成为活动字符。

```
\obeylines
```

所有特殊字符的类别码都改为 12（其他）。

```
\let\do\@makeother \dospecials
```

我们切换到要使用的字体。

```
\verbatim@font
```

为了避免在标签盒子之后断行，我们去掉列表宏在那里放置的惩罚值：又一次用到了\unpenalty！

```
\everypar \expandafter{\the\everypar \unpenalty}}
```

\verbatim
\verbatim*

现在定义顶层宏。\verbatim 略有改动：做好设置后调用\verbatim@start。这一切都在一个组内完成，这样\verbatim 就可以不通过\begin 而直接使用。

```
\def\verbatim{\begingroup\@verbatim \frenchspacing\@vobeyspaces
              \verbatim@start}
```

\verbatim* 按类似方式定义。

```
\@namedef{verbatim*}{\begingroup\@verbatim
  \@setupverbvisiblespace\@vobeyspaces
  \verbatim@start}
```

\endverbatim
\endverbatim* 要结束 verbatim 和 verbatim* 环境，只需结束\@verbatim 中启动的 trivlist 环境并关闭相应的组。

```
\def\endverbatim{\endtrivlist\endgroup\@doendpe}
\expandafter\let\csname endverbatim*\endcsname =\endverbatim
```

### 3.3 comment 环境

\comment
\endcomment \comment 宏与\verbatim* 类似。不过我们不需要切换字体或设置诸如\parindent 、\parskip 之类的特殊排版参数。只需把所有特殊字符的类别码设为 12（其他），把 ^^M（行尾字符）的类别码设为 13（活动）。后者是下面内部宏进行宏参数定界匹配所需要的。与\verbatim 、\verbatim* 宏的默认定义不同，我们把\verbatim@addtoline 定义为丢弃其参数，把\verbatim@processline 、\verbatim@startline、\verbatim@finish 定义为空操作。然后调用\verbatim@ 。但我们做的第一件事是调用\@bsphack ，使该环境对间距完全没有影响。

```
\def\comment{\@bsphack
             \let\do\@makeother\dospecials\catcode`\^^M\active
             \let\verbatim@startline\relax
             \let\verbatim@addtoline\@gobble
             \let\verbatim@processline\relax
             \let\verbatim@finish\relax
             \verbatim@}
```

\endcomment 非常简单：它只调用\@esphack 来处理间距。\end 宏关闭组，因此负责恢复我们所做的一切更改。

```
\let\endcomment=\@esphack
```

### 3.4 主循环

下面是最巧妙的部分：在定义这些宏时，我们不仅需要以正常类别码使用特殊字符\ 、{ 和} ，还需要以类别码 12（其他）使用它们。我们通过下面的技巧实现：先告诉 TEX，\、{ 和} 分别是! 、[ 和] 的小写形式。然后把每个应该以类别码 12 读取的\ 、{、} 分别替换为! 、[、]，把整串记号交给\lowercase ，因为我们知道这个原语不会改变类别码!

但首先我们必须确保!、[、] 本身有正确的类别码!为了允许对这些类别码做特殊设置，我们把它们的设置隐藏在宏\vrb@catcodes 中。如果它已经定义，就跳过我们的新定义。

```
\@ifundefined{vrb@catcodes}%
 {\def\vrb@catcodes{%
    \catcode`\!12\catcode`\[12\catcode`\]12}}{}
```

这个技巧使我们的代码可以用于其他类别码生效的场合。

我们启动一个组，使类别码改动局部化。

```
\begingroup
 \vrb@catcodes
 \lccode`\!=`\\ \lccode`\[=`\{ \lccode`\]=`\}
```

我们还需要把行尾字符 ^^M 作为活动字符。如果直接写\catcode `\^^M=\active，那么在下面每个宏定义行的末尾都会出现一个不想要的活动行尾字符。因此我们用与上面相同的技巧：用波浪号 ~ 代替 ^^M，假装后者是前者的小写形式。因此我们现在必须确保波浪号字符有类别码 13（活动）。

```
 \catcode`\~=\active \lccode`\~=`\^^M
```

使用\lowercase 原语带来一个问题：下面的代码中需要使用大写字母 C，并且必须保持其大小写。因此我们添加如下命令：

```
 \lccode`\C=`\C
```

现在开始交给\lowercase 的记号表。我们用下面这个小技巧（由 Bernd Raichle 提出）：交给\lowercase 的记号表中第一个记号是\endgroup 原语。这意味着 TEX 在\lowercase 完成操作后立即处理它，从而结束上面\begingroup 启动的组。这样就避免了所有宏的全局定义。

```
 \lowercase{\endgroup
```

\verbatim@start

\verbatim@start 的作用是检查\begin{verbatim} 同一行上是否还有其他字符，并把它们当作单独一行处理。另一方面，如果当前行上没有剩余字符，我们只会遇到一个行尾字符。\verbatim@start 通过先抓取后面的字符（即其参数）来完成任务。然后把这个参数与活动的行尾字符 ^^M 比较。

```
   \def\verbatim@start#1{%
     \verbatim@startline
     \if\noexpand#1\noexpand~%
```

如果为真，我们把控制权交给\verbatim@ 来处理下一行。我们用\next 作为继续工作的宏。

```
       \let\next\verbatim@
```

否则，我们把\next 定义为展开为对\verbatim@ 的调用，后面紧跟刚读入的字符，使它被重新插回文本。这意味着该行上剩余的字符被当作单独一行处理。

```
73      \else \def\next{\verbatim@#1}\fi
```

Finally we call \next.

```
74      \next}%
```

\verbatim@ The three macros \verbatim@, \verbatim@@, 三个宏\verbatim@ 、\verbatim@@、\verbatim@@@ 构成 verbatim 环境的" 主循环"。\verbatim@ 的作用是恰好读取一行输入。\verbatim@@ 和\verbatim@@@ 协同工作，找出该行中是否出现四个字符\end （均为类别码 12（其他））。如果出现，\verbatim@@@ 会调用\verbatim@test 检查这个\end 是否是\end{verbatim} 的一部分，如果是就结束环境。否则我们就像什么都没发生一样继续。下面看\verbatim@ 的定义：

```
75    \def\verbatim@#1~{\verbatim@@#1!end\@nil}%
```

注意：在这个宏的定义真正生效之前，! 字符已经被管辖本代码的\lowercase 原语替换为一个类别码 12（其他）的\ 。也就是说，它取一行，在后面放上\end （四个字符记号）和\@nil （一个控制序列记号）作为定界符，然后调用\verbatim@@ 。

\verbatim@@ \verbatim@@ 把直到下一次出现四个字符\end 之前的所有内容作为其参数。

```
76    \def\verbatim@@#1!end{%
```

也就是说：如果原行中没有出现它们，那么参数 #1 就是整行输入，下一个要处理的记号是\@nil 。但是，如果四个字符\end 是原行的一部分，那么 #1 由\end 前面的字符组成，下一个记号是后面的字符（永远记住该行被加长了五个记号）。无论 #1 是什么，它都是 verbatim 文本，所以 #1 被添加到当前正在构建的行中。

```
77      \verbatim@addtoline{#1}%
```

输入流中的下一个记号对我们特别重要。因此在调用\verbatim@@@ 之前，\futurelet 把\next 定义为等于该记号。

```
78      \futurelet\next\verbatim@@@}%
```

\verbatim@@@ \verbatim@@@ 现在读取当前行上剩余的记号，直到最后的\@nil 记号。

```
79    \def\verbatim@@@#1\@nil{%
```

如果上述两种情况中的第一种发生，即该行上没有\end 字符，那么 #1 为空，且\next 等于\@nil 。这很容易检查。

```
80      \ifx\next\@nil
```

如果为真，这就是一个简单行。我们处理迄今累积的行来结束它。然后准备读取下一行。

```
\verbatim@processline
\verbatim@startline
\let\next\verbatim@
```

否则我们必须检查这些\end 记号后面跟着什么。

```
\else
```

在继续之前，最好停下来想一下我们在哪儿：我们刚读完四个字符记号\end，现在必须检查后面是否跟着环境名（用花括号括起来）。为此我们定义一个叫\@tempa 的宏，它恰好读取一个字符并决定下一步做什么。这个宏应该做下面的事：跳过空格，直到遇到左花括号或行尾。但重要的是记住跳过了哪些字符。\end⟨可选空格⟩{ 这些字符可能是 verbatim 文本的一部分，即这些字符必须被打印。

例如，假设当前行包含

```
␣␣␣␣␣\end␣{AVeryLongEnvironmentName}
```

正如我们很快会看到的，这里实现的扫描机制在读完右花括号之前不会发现这是要打印的文本。因此我们需要一种累积已读字符的方法，以便在必要时把它们重新插回。记号寄存器\@temptokena 用于此目的。

在做这件事之前，我们必须先去掉行末多余的\end 记号。为此我们定义一个临时宏，其参数由\end\@nil （四个字符记号和一个控制序列记号）界定，用于下面处理该行剩余部分——先在该行末尾追加一个\@nil 记号。（注意这个记号永远不可能出现在 #1 中。）我们用下面的\@tempa 定义来获取该行（第一个\end 之后）的剩余部分。

```
\def\@tempa##1!end\@nil{##1}%
```

我们前面已经提到，用记号寄存器\@temptokena 来记住跳过的字符，以防再次需要它们。我们用在\@tempa 调用中丢弃的那个\end 来初始化它。

```
\@temptokena{!end}%
```

现在我们要调用\verbatim@test 来处理当前行上剩余的字符。但等一下：我们不能直接调用这个宏，因为我们已经读完了整行。因此我们必须先展开宏\@tempa，把这些字符重新插回到\verbatim@test 记号之后。末尾追加一个 ^^M 字符表示行尾。（记住这个字符伪装成波浪号。）

```
\def\next{\expandafter\verbatim@test\@tempa#1\@nil~}%
```

差不多就这些了，但我们还需要调用\next 来完成工作。

```
\fi \next}%
```

\verbatim@test 我们定义\verbatim@test 来依次检查每个记号。

```
\def\verbatim@test#1{%
```

首先，我们把\next 设为\verbatim@test ，以防这个宏需要递归调用来跳过空格。

```
    \let\next\verbatim@test
```

我们必须区分四种情况：

1. 下一个记号是 ^^M，即我们到达了行尾。这意味着没有发现任何特殊内容。注意我们在下面的比较中使用\if ，这样字符的类别码就无关紧要了。

```
      \if\noexpand#1\noexpand~%
```

   我们把记号寄存器\@temptokena 中累积的字符添加到当前行。由于\verbatim@addtoline 不展开其参数，我们必须在此处展开。然后用\let 把\next 设为\verbatim@ ，准备读取下一行。

```
        \expandafter\verbatim@addtoline
          \expandafter{\the\@temptokena}%
        \verbatim@processline
        \verbatim@startline
        \let\next\verbatim@
```

2. 后面是空格字符。这是允许的，所以我们把它添加到\@temptokena 并继续。

```
      \else \if\noexpand#1
        \@temptokena\expandafter{\the\@temptokena#1}%
```

3. 后面是左花括号。这是最有趣的情况。我们现在必须收集字符，直到读到右花括号，并检查它们是否构成环境名。这将由\verbatim@testend 完成，所以这里我们把\next 设为这个宏。我们将再次逐字符处理该行剩余部分。构成环境名的字符将累积在\@tempc 中。我们把这个宏初始化为展开为空。

```
      \else \if\noexpand#1\noexpand[%
        \let\@tempc\@empty
        \let\next\verbatim@testend
```

   注意：当这个宏被定义时，[ 字符将是一个 {。

4. 任何其他字符都意味着这个\end 是 verbatim 文本的一部分。把这些字符添加到当前行，并准备调用\verbatim@ 来处理该行剩余部分。

```
102                 \else
103                   \expandafter\verbatim@addtoline
104                     \expandafter{\the\@temptokena}%
105                   \def\next{\verbatim@#1}%
106                 \fi\fi\fi
```

这个宏做的最后一件事是调用\next 来继续处理。

```
107           \next}%
```

\verbatim@testend 当看到\end ⟨可选空格⟩{ 时调用\verbatim@testend 。它的任务是扫描直到下一个} 的所有内容，并调用\verbatim@@testend 。如果没找到} ，它必须重新插回所读字符并返回\verbatim@ 。下面的定义与\verbatim@test 类似：它取下一个字符并决定做什么。

```
108     \def\verbatim@testend#1{%
```

同样，我们有四种情况：

1. ^^M：由于当前行中没有找到} ，把这些字符添加到缓冲区。为避免展开\@temptokena 和\@tempc 的复杂构造，我们分两步做。然后继续用\verbatim@ 处理下一行。

```
109               \if\noexpand#1\noexpand~%
110                 \expandafter\verbatim@addtoline
111                   \expandafter{\the\@temptokena[}%
112                 \expandafter\verbatim@addtoline
113                   \expandafter{\@tempc}%
114                 \verbatim@processline
115                 \verbatim@startline
116                 \let\next\verbatim@
```

2. }：调用\verbatim@@testend 检查这是否是正确的环境名。

```
117               \else\if\noexpand#1\noexpand]%
118                 \let\next\verbatim@@testend
```

3. \：这个字符不应出现在环境名中。因此我们停止收集字符。原则上，同样的理由也适用于其他字符，例如 {。不过\ 是一种特殊情况，因为它可能是\end 的第一个字符。这意味着我们必须重新寻找\end{ ⟨环境名⟩}。注意我们在! 前面加了一个\noexpand 原语，以防止它是一个活动字符。

```
119               \else\if\noexpand#1\noexpand!%
120                 \expandafter\verbatim@addtoline
121                   \expandafter{\the\@temptokena[}%
```

```
122                 \expandafter\verbatim@addtoline
123                   \expandafter{\@tempc}%
124                 \def\next{\verbatim@!}%
```

4. 任何其他字符：收集它并继续。我们不能用\edef 来定义\@tempc ，因为它的替换文本可能包含活动字符记号。

```
125               \else \expandafter\def\expandafter\@tempc\expandafter
126                 {\@tempc#1}\fi\fi\fi
```

和前面一样，这个宏最后通过递归调用来处理下一个字符（如果合适的话）。

```
127         \next}%
```

\verbatim@@testend 与前面的宏不同，\verbatim@@testend 很简单：它只需检查\begin{ …} 是否与对应的\end{ …} 匹配。

```
128     \def\verbatim@@testend{%
```

我们再次用\next 来定义要做的事。记住当前环境名存放在\@currenvir 中，由\verbatim@testend 累积的字符在\@tempc 中。所以我们只需比较这两者，如果匹配就准备执行\end{ ⟨当前环境⟩} 宏。在做这件事之前，我们调用\verbatim@finish 来处理最后一行。我们通过\edef 定义\next ，这样\@currenvir 就会被其展开替换。因此我们需要\noexpand 来抑制此时\end 的展开。

```
129       \ifx\@tempc\@currenvir
130         \verbatim@finish
131         \edef\next{\noexpand\end{\@currenvir}%
```

没有这个技巧，\end 命令将无法正确检查其参数是否与当前环境名匹配，你会得到一条有趣的 LaTeX 错误信息，例如：

```
! \begin{verbatim*} ended by \end{verbatim*}.
```

但是，那一行上剩余的字符怎么办？我们调用\verbatim@rescan 来处理。它的第一个参数是刚结束的环境名（以防我们还需要它）。\verbatim@rescan 把待重新处理的字符列表作为第二个参数。（这个记号表是由\verbatim@@@ 插入到当前宏之后的。）由于我们仍在\edef 中，我们用\noexpand 保护它。

```
132                     \noexpand\verbatim@rescan{\@currenvir}}%
```

如果名称不匹配,我们把迄今读到的所有字符重新插回,并准备调用\verbatim@来处理该行剩余部分。

```
133       \else
134         \expandafter\verbatim@addtoline
135           \expandafter{\the\@temptokena[}%
```

```
            \expandafter\verbatim@addtoline
              \expandafter{\@tempc]}%
          \let\next\verbatim@
        \fi
```

最后我们调用\next 。

```
        \next}%
```

\verbatim@rescan 原则上，\verbatim@rescan 可以用来分析\end{...} 命令之后剩余的字符，并假装这些字符是在” 标准” 类别码生效下被” 正确” 读取的。[1] 但这并非总是可行（当该行剩余部分中有不匹配的花括号时）。此外，我们认为这不值得费力：在 verbatim 或 verbatim* 环境之后，输出反正会开始新行，而\end{comment} 很容易单独放在一行上。因此这里没有理由出现任何文本。对于确实在那里放了东西（也许是注释）的用户，我们简单地发出一条警告并丢弃它们。测试方法在 TEXbook 附录 D 第 376 页有说明。我们用 ^^M 代替那里用的! 字符，因为这个字符不可能出现在 #1 中。两个\noexpand 原语是必要的，以避免活动字符和宏的展开。

这里还应注意一个额外的微妙之处：记住我们当前正在构建的记号表，会先被\lowercase 原语处理，然后 TEX 才执行这些定义。这意味着\@warning 宏参数中的 C 字符必须受到保护，防止被改成 c。这就是我们在上面添加\lccode `\C=`\C 赋值的原因。现在我们可以结束\lowercase 的参数以及改变类别码的那个组了。

```
    \def\verbatim@rescan#1#2~{\if\noexpand~\noexpand#2~\else
        \@warning{Characters dropped after `\string\end{#1}'}\fi}}
```

## 3.5 \verbatiminput 命令

\verbatim@in@stream 我们首先分配一个输入流（从可用的 16 个输入流中）。

```
\newread\verbatim@in@stream
```

\verbatim@readfile 宏\verbatim@readfile 通过分别调用\verbatim@startline 和\verbatim@finish 宏来包围主循环。这样可以确保当文件为空或不存在时，用户仍能正确初始化和结束该命令。verbatim 环境在以空文本调用时有类似行为。

```
\def\verbatim@readfile#1{%
  \verbatim@startline
```

当找不到文件时，我们发出一条警告。

```
  \openin\verbatim@in@stream #1\relax
  \ifeof\verbatim@in@stream
```

[1] 记住它们都是以类别码 11（字母）和 12（其他）读取的，这样控制序列就不会被识别为控制序列。

```
    \typeout{No file #1.}%
  \else
```

此时我们把文件名传给\@addtofilelist ，使它出现在\listfiles 命令的输出中。此外，我们使用\ProvidesFile 在日志文件中记录一条日志项，并区分通过\verbatiminput 读入的文件与其他文件。

```
    \@addtofilelist{#1}%
    \ProvidesFile{#1}[(verbatim)]%
```

从文件读取时，关闭对行尾字符的识别是有用的。这样我们就不必从行内容中去掉空格。

```
    \expandafter\endlinechar\expandafter\m@ne
    \expandafter\verbatim@read@file
    \expandafter\endlinechar\the\endlinechar\relax
    \closein\verbatim@in@stream
  \fi
  \verbatim@finish
}
```

\verbatim@read@file 所有工作都在\verbatim@read@file 中完成。它逐行读取输入文件，并递归调用自己直到文件末尾。

```
\def\verbatim@read@file{%
  \read\verbatim@in@stream to\next
  \ifeof\verbatim@in@stream
  \else
```

对每一行，我们用该行内容调用\verbatim@addtoline 。
接着调用\verbatim@processline 。

```
    \expandafter\verbatim@addtoline\expandafter{\next}%
    \verbatim@processline
```

处理完一行后，我们在读取下一行之前调用\verbatim@startline 来重新初始化一切。

```
    \verbatim@startline
```

如果没有\expandafter ，每次调用\verbatim@read@file 都会在 TEX 的输入栈中占用空间。[2]

```
    \expandafter\verbatim@read@file
  \fi
}
```

[2]标准 TEX 在你试图读取超过约 200 行的文件时会报告溢出错误。如果 *"TeX: The Program"*§390 中的第一行代码缺失，也会发生同样的错误。

\verbatiminput \verbatiminput 首先启动一个组，使字体和类别码改动局部化。然后它根据后面是否跟星号，带附加参数调用宏\verbatim@input 。

```
\def\verbatiminput{\begingroup
```

如果是星号版本，我们标记空格和 TAB，这两个新增部分与 verbatim* 相同。

```
  \@ifstar{\verbatim@input{\@setupverbvisiblespace\@vobeyspaces}}%
          {\verbatim@input{\frenchspacing\@vobeyspaces}}}
\IfDocumentMetadataT
{
 \def\verbatiminput{\begingroup
   \@ifstar{\verbatim@input{final-code=\legacyverbatimsetup{visible}}}%
           {\verbatim@input{final-code=\legacyverbatimsetup{invisible}}}}
}
```

\verbatim@input \verbatim@input 首先用标准宏\IfFileExists 检查文件是否存在，它会把找到的文件名留在\@filef@und 中。然后像\verbatim 宏一样做好一切设置。但是，由于\@verbatim 包含对\every@verbatim 的调用，而后者可能包含一条\input 语句，会覆盖\@filef@und 的内容，我们需要先展开它来保存。使用\@swaptwoargs 使得\@filef@und 的展开结果成为\verbatim@readfile 的第二个参数。

```
\def\verbatim@input#1#2{%
  \IfFileExists {#2}{%
    \expandafter\@swaptwoargs\expandafter
      {\expandafter{\@filef@und}}%
      {\@verbatim #1\relax
```

然后它读入文件，结束由\@verbatim 启动的 trivlist 环境，并关闭组。这会把一切恢复到正常设置。

```
        \verbatim@readfile}%
      \endtrivlist\endgroup\@doendpe}%
```

如果找不到文件，会打印一条多少有点用的信息。最后的\endgroup 是为了关闭上面\verbatiminput 中启动的组。

```
   {\typeout {No file #2.}\endgroup}}
\IfDocumentMetadataT
{
  \def\verbatim@input#1#2{%
    \IfFileExists {#2}{%
      \expandafter\@swaptwoargs\expandafter
       {\expandafter{\@filef@und}}%
```

```
192          {%
193           \SimpleBlockEnv{verbatim} {#1}\obeyedline
194           \verbatim@readfile}
195           \BlockEnvEnd\endgroup
196         }%
197      {\typeout {No file #2.}\endgroup}}
198  }
```

199 ⟨/package⟩

## 3.6 把 verbbatim 文本送入参数

实现这一点的一种方法是定义一个宏（命令），其展开结果就是所需的 verbatim 文本。这个命令随后可以在任何需要 verbatim 文本的地方使用。它可以用在参数中，甚至是移动参数中，但它是脆弱的（至少这里的版本是）。

下面这段代码声称提供了这个功能。它是我（Chris）约两年前所写东西的大幅修订版。也许还需要进一步修订。

它只打算作为`\verb` 的扩展，而不是 `verbatim` 环境的扩展。因此，它或许应该以对`\verb` 最合适的方式来处理行尾。

`\newverbtext` 这是产生一个新宏的命令，其展开结果是 verbatim 文本。这个命令本身当然不能用在参数中！它的用法如下：

```
\newverbtext{\myverb}"^%{ &~_\}}@ #"
```

界定 verbatim 文本的规则与`\verb` 相同。

200 ⟨*verbtext⟩

```
201 \def \newverbtext {%
202   \@ifstar {\@tempswatrue \@verbtext }{\@tempswafalse \@verbtext *}%
203 }
```

我认为在这里使用一个临时开关是安全的：如果不安全，可以用合适的`\let` 。

```
204 \def \@verbtext *#1#2{%
205   \begingroup
206     \let\do\@makeother \dospecials
207     \let\do\do@noligs \verbatim@nolig@list
208     \@vobeyspaces
209     \catcode`#2\active
210     \catcode`~\active
211     \lccode`\~`#2%
212     \lowercase
```

我们在这里用一个临时宏和一个技巧，使命令本身的定义可以在组内完成，并且是局部定义（也许有更好的方法来实现这一点）。

```
    {\def \@tempa ##1~%
          {\whitespaces
```

如果这些\noexpand 改成\noexpand\protect\noexpand ，会不会让这些东西变得健壮？

```
           \edef #1{\noexpand \@verbtextmcheck
                    \bgroup
                    \if@tempswa
                      \noexpand \visiblespaces
                    \fi
                    \noexpand \@verbtextsetup
                    ##1%
                    \egroup}%
           }%
     \expandafter\endgroup\@tempa
    }
}
```

这为当前模式设置正确类型的组：它绝不能在定义时被展开！

```
\def \@verbtextmcheck {%
  \relax\ifmmode
          \hbox
        \else
          \leavevmode
          \null
        \fi
}
```

这里包含其他一些在定义期间不应展开的东西。

```
\def \@verbtextsetup {%
  \frenchspacing
  \verbatim@font
  \verbtextstyle
}
```

命令\verbtextstyle 是一个文档级钩子，可用于覆盖用\newverbtext 命令定义的命令的预定义排版处理。

\visiblespaces 和\whitespaces 是这个钩子可能取值的例子。

```
\let \verbtextstyle \relax
\def \visiblespaces {\chardef \  32\relax}
\def \whitespaces {\let \ \@@space}
```

```
\let \@@space \ %
⟨/verbtext⟩
```

# 4 测试实现

为了测试实现并做演示，我们提供了一个额外文件。它可以通过条件 'testdriver' 抽取。它使用一个单独分发的小输入文件 'verbtest.tst'。我们再次使用单独的 '+' 守卫。

```
⟨*testdriver⟩
\documentclass{article}

\usepackage{verbatim}

\newenvironment{myverbatim}%
   {\endgraf\noindent MYVERBATIM:\endgraf\verbatim}%
   {\endverbatim}

\makeatletter

\newenvironment{verbatimlisting}[1]%
 {\def\verbatim@startline{\input{#1}%
    \def\verbatim@startline{\verbatim@line{}}%
    \verbatim@startline}%
  \verbatim}{\endverbatim}

\newwrite\verbatim@out

\newenvironment{verbatimwrite}[1]%
 {\@bsphack
  \immediate\openout \verbatim@out #1
  \let\do\@makeother\dospecials\catcode`\^^M\active
  \def\verbatim@processline{%
    \immediate\write\verbatim@out{\the\verbatim@line}}%
  \verbatim@start}%
 {\immediate\closeout\verbatim@out\@esphack}

\makeatother

\addtolength{\textwidth}{30pt}

\begin{document}
```

```
278
279 \typeout{}
280 \typeout{===> Expect ``characters dropped''
281          warning messages in this test! <====}
282 \typeout{}
283
284 Text Text Text Text Text Text Text Text Text Text Text
285 Text Text Text Text Text Text Text Text Text Text Text
286 Text Text Text Text Text Text Text Text Text Text Text
287   \begin{verbatim}
288     test
289     \end{verbatim*}
290     test
291     \end{verbatim
292     test of ligatures: <`!`?`>
293     \endverbatim
294     test
295     \end  verbatim
296     test
297     test of end of line:
298     \end
299     {verbatim}
300   \end{verbatim} Further text to be typeset: $\alpha$.
301 Text Text Text Text Text Text Text Text Text Text Text
302 Text Text Text Text Text Text Text Text Text Text Text
303 Text Text Text Text Text Text Text Text Text Text Text
304   \begin{verbatim*}
305     test
306     test
307   \end {verbatim*}
308 Text Text Text Text Text Text Text Text Text Text Text
309 Text Text Text Text Text Text Text Text Text Text Text
310 Text Text Text Text Text Text Text Text Text Text Text
311   \begin{verbatim*}  bla bla
312     test
313     test
314   \end {verbatim*}
315 Text Text Text Text Text Text Text Text Text Text Text
316 Text Text Text Text Text Text Text Text Text Text Text
317 Text Text Text Text Text Text Text Text Text Text Text
318 Text Text Text Text Text Text Text Text Text Text Text
319
```

```
First of Chris Rowley's fiendish tests:
\begin{verbatim}
the double end test<text>
\end\end{verbatim}  or even \end \end{verbatim}
%
%not \end\ended??
%\end{verbatim}

Another of Chris' devils:
\begin{verbatim}
the single brace test<text>
\end{not the end\end{verbatim}
%
%not \end}ed??
%\end{verbatim}
Back to my own tests:
  \begin{myverbatim}
    test
    test
  \end {myverbatim} rest of line
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text

Test of empty verbatim:
\begin{verbatim}
\end{verbatim}
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text
  \begin {verbatimlisting}{verbtest.tex}
    Additional verbatim text
      ...
  \end{verbatimlisting}
And here for listing a file:
  \verbatiminput{verbtest.tex}
And again, with explicit spaces:
  \verbatiminput*{verbtest.tex}
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text
  \begin{comment}
```

```
    test
    \end{verbatim*}
    test
    \end {comment
    test
    \endverbatim
    test
    \end  verbatim
    test
  \end {comment} Further text to be typeset: $\alpha$.
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text
  \begin{comment}  bla bla
    test
    test
  \end {comment}
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text
Text Text Text Text Text Text Text Text Text Text Text

\begin{verbatimwrite}{verbtest.txt}
asfa<fa<df
sdfsdfasd
asdfa<fsa
\end{verbatimwrite}

\end{document}
```

⟨/testdriver⟩